SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively.*(BL3)*

Assessment Tool Weightage: 50%

QUESTIONS: 3 **Duration : 1 Hour**
Total Marks:30

Annexure A

Constraint-Based Problem Solving

Code of Conduct:

I s.prajith kumar (1912260177) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

s.prajith

Annexure A

Constraint-Based Problem Solving

1. A hospital needs to assign 3 doctors (D1, D2, D3) to 3 shifts (Morning, Afternoon, Night) such that:

   Constraints:

   i. D1 cannot work Night shift

   ii. D2 must work before D3 (Morning < Afternoon < Night)

   iii. Only one doctor per shift

   iv. D3 cannot work Morning

   v. All doctors must be assigned exactly one shift

   Apply Backtracking Search to find a valid assignment with all intermediate steps and constraint checks. State the final valid schedule

2. A robot operates in a grid environment (5×5). It must move from Start (S) to Goal (G). Obstacles block certain paths.

   Constraints:

   i. Movement allowed: Up, Down, Left, Right
   ii. Cost of each move = 1
   iii. Cannot pass through obstacles
   iv. Use Manhattan Distance heuristic

   Using above constraints and BFS Search Algorithm Show step-by-step node expansion and priority queue updates and determine the optimal path and cost.

3. An autonomous rescue robot is deployed in a disaster-affected building to locate and rescue survivors. The robot operates in a partially observable environment where some paths are blocked, and it must make decisions with limited information.

The building is represented as a grid of rooms. The robot starts at position S and must reach a survivor located at G.

Constraints:

- The robot can move up, down, left, right
- Some cells are blocked (obstacles) and cannot be traversed
- The robot has limited sensing capability (can only observe adjacent cells)
- Each movement has a cost of 1, but entering risky zones has an additional cost of 2
- The robot must minimize total cost while ensuring safe navigation
- The robot must update its knowledge dynamically (online search scenario)

Tasks:

a) Analyze all the given constraints and explain how they affect the robot's decision-making process.

b) Develop a solution approach using an appropriate search strategy (e.g., Uniform Cost Search / Online Search Agent). Clearly explain the steps involved.
c) Demonstrate how the robot applies AI concepts such as:

- Intelligent agents
- Rationality
- Environment type

d) Justify why your chosen approach is the most suitable for solving this problem under the given constraints.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

1. **Backtracking Search**

Backtracking Search is a systematic search technique used to solve Constraint Satisfaction Problems (CSPs). It assigns values to variables one at a time and checks the constraints after each assignment. If an assignment violates any constraint, that choice is rejected and the algorithm goes back (backtracks) to try another possible assignment.

Given Problem Doctors: D1, D2, D3 Shifts:

- Morning
- Afternoon
- Night

Constraints

1. D1 cannot work the Night shift.
2. D2 must work before D3.
3. Only one doctor can be assigned to each shift.
4. D3 cannot work the Morning shift.
5. Every doctor must be assigned exactly one shift.

Step-by-Step Backtracking Step 1:
Assign D1

Try:
D1 → Morning Constraint
check:

- D1 is not assigned Night → Satisfied.
- No other doctor has been assigned → Satisfied. So, keep:

D1 = Morning

Step 2: Assign D2

D2 must work before D3. Try:
D2 → Afternoon Constraint
check:

- D2 is in Afternoon.
- D3 must therefore be after Afternoon.
- The only remaining later shift is Night. So:

D2 = Afternoon

Step 3: Assign D3
The remaining shift is Night. Try:
D3 → Night Constraint check:

- D3 is not assigned Morning → Satisfied.
- D2 is before D3: Afternoon < Night → Satisfied.
- Only one doctor occupies each shift → Satisfied.
- All doctors have exactly one shift → Satisfied. Therefore, this assignment is valid.

Final Valid Schedule

Doctor	**Shift**
D1	Morning
D2	Afternoon
D3	Night

Constraint Verification

- D1 ≠ Night → **Satisfied**
- D2 < D3 → **Afternoon < Night → Satisfied**
- One doctor per shift → **Satisfied**
- D3 ≠ Morning → **Satisfied**

- All doctors assigned exactly one shift → **Satisfied Conclusion**

Using Backtracking Search, a valid schedule is:

D1 → Morning, D2 → Afternoon, D3 → Night

All the given constraints are satisfied, so this is a valid solution.

2. BFS Search for Robot Navigation

Given Problem

The robot operates in a **5×5 grid** and must move from the **Start (S)** position to the **Goal (G)** position.

The following constraints are given:

1. The robot can move **Up, Down, Left, or Right**.
2. The cost of every movement is **1**.
3. The robot cannot move through an obstacle.
4. Manhattan Distance is given as the heuristic: **Manhattan Distance = $|x_1 - x_2| + |y_1 - y_2|$ Breadth-First Search (BFS)**

BFS explores the grid level by level. It first visits the starting node, then all nodes one step away, followed by nodes two steps away, and so on. Because every movement has the same cost of **1**, BFS finds a path with the minimum number of moves.

Step-by-Step Procedure Step 1: Start at S

Place the starting position **S** in the queue.

Queue:

[S]

Visited nodes:

Step 2: Expand S

Check the four possible directions:

- Up
- Down
- Left
- Right

Any position that is outside the grid or contains an obstacle is rejected.

The valid neighbouring positions are added to the queue.

Step 3: Expand the First Node in the Queue

Remove the first node from the queue and examine its valid neighbours.

For every neighbour:

- Check whether it is inside the 5×5 grid.
- Check whether it is an obstacle.
- Check whether it has already been visited.
- If valid, add it to the queue.

Step 4: Continue BFS

Continue expanding nodes in FIFO order:
Level 0 → Level 1 → Level 2 → Level 3 → … The search stops when the **Goal (G)** is reached. **Priority Queue / Search Order**

Since all movements have a cost of 1, BFS expands nodes according to their depth from S. For example:

Level	**Meaning**
0	S

Level	Meaning
1	Nodes one move from S
2	Nodes two moves from S
3	Nodes three moves from S
…	…
Goal level	G is reached

The Manhattan distance can be calculated as:

$h(n) = |x_n - xG| + |y_n - yG|$

This gives the number of horizontal and vertical steps between the current node and the goal, ignoring obstacles.

Optimal Path

When BFS reaches G, trace the parent of G backward until reaching S. This gives the path:

S → … → … → G

The optimal cost is:

Cost = Number of movements × 1

Therefore:

Optimal Cost = Number of moves in the BFS path Conclusion

BFS is suitable because every movement has the same cost of
1. It explores the grid systematically and guarantees the shortest path in terms of the number of moves, provided the grid and obstacles are fixed.

Note: The exact node expansion order, queue contents, optimal path, and numerical cost require the missing **5×5 grid containing S, G, and the obstacles**.

3. Autonomous Rescue Robot

(a) Analysis of Constraints

The rescue robot operates in a partially observable disaster environment. The following constraints affect its decision-making:

1. **Movement constraint:**
   The robot can move only Up, Down, Left, and Right. Therefore, it cannot move diagonally.
2. **Obstacle constraint:**
   Some cells are blocked. The robot cannot enter these cells and must find another safe route.
3. **Limited sensing:**
   The robot can observe only adjacent cells. It does not have complete information about the entire building.
4. **Movement cost:**

   Every normal movement has a cost of 1.
5. **Risky-zone cost:**
   Entering a risky zone adds an additional cost of 2. Therefore, entering a risky zone has a total movement cost of 3.
6. **Minimum-cost requirement:**
   The robot must reach the survivor while minimizing the total cost.
7. **Dynamic knowledge:**
   The robot must update its knowledge as it discovers new information. This makes the problem suitable for an online search approach.

(b) Suitable Search Strategy

A suitable approach is **Uniform Cost Search (UCS) combined with an Online Search Agent**.

Step 1: The robot starts at position S.
Step 2: It observes the adjacent cells using its limited sensing capability.
Step 3: The robot identifies possible movements and their costs.

- Normal cell → cost = 1
- Risky cell → cost = 1 + 2 = 3
- Obstacle → cannot be entered

Step 4: The robot places possible paths in a priority queue according to their accumulated cost.
Step 5: The path with the lowest total cost is selected for expansion.
Step 6: When the robot moves to a new location, it observes the newly available information.
Step 7: The robot updates its knowledge of obstacles and risky areas.
Step 8: The search continues until the robot reaches G. The general cost calculation is:
Total Cost = Normal Movement Costs + Risky-Zone Additional Costs

For example, if a path contains 5 normal movements and 1

risky-zone entry:

Total Cost = (5 × 1) + (1 × 2) = 7
The robot should select a safe path with the minimum total cost.

(c) Application of AI Concepts

1. Intelligent Agent

The rescue robot is an intelligent agent because it:

- Perceives its environment using sensors.
- Processes the available information.
- Selects an appropriate action.
- Moves toward the survivor.
- Updates its knowledge based on new observations. The robot follows the cycle:

Perceive → Decide → Act → Update Knowledge

2. Rationality

The robot is a rational agent because it chooses actions that help achieve its goal while minimizing cost and maintaining safe navigation.
It should avoid unnecessary risky areas when a safer and lower-cost route is available.

3. Environment Type

The environment is:

- **Partially observable** – the robot can observe only adjacent cells.
- **Dynamic** – the robot updates its knowledge as it discovers new information.
- **Sequential** – each movement affects the following decisions.
- **Discrete** – the robot moves between individual grid cells.
- **Cost-based** – different movements can have different costs because of risky zones.

(d) Justification of the Approach

Uniform Cost Search is suitable because the objective is to find a path with the **minimum total cost**, rather than simply the minimum number of movements.

A normal movement costs 1, while entering a risky zone adds an additional cost of 2. Therefore, different paths can have different total costs.
UCS expands the lowest-cost path first, making it appropriate for finding a minimum-cost solution.
Because the robot cannot see the complete environment in advance, an **Online Search Agent** is also appropriate. The robot can:

1. Observe nearby cells.
2. Update its knowledge.
3. Select the best available action.
4. Move to the next cell.
5. Repeat the process until G is reached.

Final Conclusion
The combination of **Uniform Cost Search and Online Search** is appropriate for the rescue robot because it considers movement costs, avoids obstacles, handles risky zones, and allows the robot to update its knowledge while navigating a partially observable environment.